

Computer Organization and Architecture

Complete Semester Exam Notes — Units 1 to 5

Contents

Unit 1: Representation of Information and Basic Building Blocks	3
1.1 Introduction to Computer	3
1.2 Computer Hardware Generations	3
1.3 Number Systems	4
1.4 Character Codes	4
1.5 Logic Gates	5
1.6 Boolean Algebra	6
1.7 K-Map (Karnaugh Map) Simplification	6
1.8 Combinational Circuits — Adders & Subtractors	7
1.9 Decoders, Encoders, Multiplexers, Demultiplexers	8
1.10 Carry Look-Ahead Adder (CLA)	8
1.11 Combinational Logic Design	9
1.12 Flip-Flops (Sequential Building Blocks)	9
1.13 Registers	10
1.14 Counters	10
1.15 ALU (Arithmetic Logic Unit)	11
1.16 Micro-Operations	11
1.17 ALU Chip	11
1.18 Faster Algorithms — Multiplication & Division	11
Unit 2: Basic Organization	13
2.1 Von Neumann Architecture	13
2.2 Operational Flow Chart of a Computer	13
2.3 Instruction Cycle	13
2.4 Organization of the Central Processing Unit (CPU)	13
2.5 Control Unit Design: Hardwired vs Micro-Programmed	14
2.6 CPU Organization Types (based on registers)	14
2.7 Addressing Modes	15
2.8 Instruction Formats	16
2.9 Data Transfer and Manipulation Instructions	16
2.10 I/O Organization (Overview)	16
2.11 Bus Architecture	17
2.12 Programming (CPU) Registers — Summary	17
Unit 3: Memory Organization	18

3.1 Memory Hierarchy	18
3.2 Main Memory (RAM / ROM)	18
3.3 Auxiliary (Secondary) Memory	19
3.4 Associative Memory (Content Addressable Memory – CAM)	19
3.5 Cache Memory	19
3.6 Virtual Memory	20
3.7 Memory Management Hardware	21
3.8 Magnetic Disk and Its Performance	21
3.9 Magnetic Tape	21
Unit 4: I/O Organization	22
4.1 Peripheral Devices	22
4.2 I/O Interface	22
4.3 Modes of Data Transfer	22
4.4 Priority Interrupt	23
4.5 Direct Memory Access (DMA) — Detailed	23
4.6 Input Output Processor (IOP)	24
4.7 Serial Communication	24
4.8 I/O Controllers	25
4.9 Asynchronous Data Transfer	25
Unit 5: Process Organization	26
5.1 Basic Concept of Microprocessors	26
5.2 Assembly Instruction Set (8085) — Common Categories	26
5.3 Assembly Language Programs (8085)	27
5.4 Introductory Concept of Pipelining	29
5.5 Flynn’s Classification	29
5.6 Feng’s Classification	30
5.7 Parallel Architecture Classification (Based on Memory Access)	30
5.8 Concept of Multi-Core Architecture	31

Unit 1: Representation of Information and Basic Building Blocks

1.1 Introduction to Computer

A **computer** is an electronic device that accepts data (input), processes it according to a set of instructions (program), and produces meaningful results (output). It works on the basic cycle: **Input → Process → Output → Storage**.

Characteristics of a computer: Speed, Accuracy, Diligence, Storage capability, Versatility, Automation.

Basic block diagram (functional units):

- **Input Unit** – accepts data/instructions (keyboard, mouse, scanner)
- **CPU (Central Processing Unit)** – ALU + CU + Registers; the “brain” of the computer
- **Memory Unit** – stores data/instructions (Primary and Secondary memory)
- **Output Unit** – presents processed results (monitor, printer)

Input Unit → Memory Unit ↔ CPU (CU + ALU) → Output Unit

1.2 Computer Hardware Generations

Generation	Period	Technology Used	Key Features
1st	1940–1956	Vacuum tubes	Huge size, high power consumption, machine language, e.g., ENIAC, UNIVAC
2nd	1956–1963	Transistors	Smaller, faster, less heat, assembly language, e.g., IBM 1401
3rd	1964–1971	Integrated Circuits (IC)	High-level languages (COBOL, FORTRAN), OS introduced
4th	1971–present	Microprocessors (VLSI)	Personal computers, GUI, e.g., Intel 4004, IBM PC
5th	Present–future	AI, ULSI, parallel processing	Natural language processing, robotics, quantum computing

1.3 Number Systems

A number system defines how numbers are represented using a set of symbols (digits) and a **base (radix)**.

System	Base	Digits Used
Binary	2	0, 1
Octal	8	0-7
Decimal	10	0-9
Hexadecimal	16	0-9, A-F

1.3.1 Conversions

Decimal → Binary/Octal/Hex: Repeated division by the base; collect remainders in reverse order.

Example: 45 (decimal) → Binary $45/2=22$ r1, $22/2=11$ r0, $11/2=5$ r1, $5/2=2$ r1, $2/2=1$ r0, $1/2=0$ r1 → **45 = (101101)₂**

Binary → Decimal: Multiply each bit by $2^{\text{(position)}}$, sum up. $(101101)_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 45$

Binary → Octal: Group bits in 3 from right (pad with 0s). $101\ 101 = 5\ 5 \rightarrow (55)_8$

Binary → Hexadecimal: Group bits in 4 from right. $0010\ 1101 = 2\ D \rightarrow (2D)_{16}$

Octal → Decimal: Multiply each octal digit by 8^{position} . **Hex → Decimal:** Multiply each hex digit by 16^{position} .

1.3.2 Binary Arithmetic

- **Addition:** $0+0=0$, $0+1=1$, $1+1=10$ (carry 1), $1+1+1=11$
- **Subtraction:** Using **1's complement** (invert all bits) or **2's complement** (1's complement + 1) for signed subtraction.
- **2's complement of a number** = invert all bits and add 1. Used to represent negative numbers and simplify subtraction (subtraction becomes addition of 2's complement).
- **Multiplication/Division:** Similar to decimal, done using shift-and-add / shift-and-subtract methods (see 1.11).

1.4 Character Codes

Character codes are binary patterns used to represent letters, digits, and symbols so a computer can process text.

1.4.1 BCD (Binary Coded Decimal)

Each decimal digit (0-9) is represented independently by a 4-bit binary code. Example: 25 (decimal) = 0010 0101 (BCD) — note this is **not** the same as pure binary of 25.

1.4.2 ASCII (American Standard Code for Information Interchange)

- 7-bit code (128 characters); extended ASCII uses 8 bits (256 characters).
- Represents English letters, digits, punctuation, and control characters.
- Example: 'A' = 65 = 1000001₂

1.4.3 EBCDIC (Extended Binary Coded Decimal Interchange Code)

- 8-bit code developed by IBM, used mainly in IBM mainframe systems.
- Can represent 256 characters, but its bit patterns differ from ASCII (not directly compatible).

Code	Bits	Max Characters	Used In
BCD	4	16 (0-9 used)	Digital calculators, early systems
ASCII	7/8	128/256	PCs, internet, most modern systems
EBCDIC	8	256	IBM mainframes

1.5 Logic Gates

Logic gates are the basic building blocks of digital circuits; they implement Boolean functions.

Gate	Symbol Meaning	Boolean Expression	Truth Table Behaviour
AND	Output HIGH only if all inputs HIGH	$Y = A \cdot B$	1 only when $A=1, B=1$
OR	Output HIGH if any input HIGH	$Y = A + B$	0 only when $A=0, B=0$
NOT	Inverter	$Y = A'$	Complements input
NAND	AND + NOT	$Y = (A \cdot B)'$	0 only when $A=1, B=1$
NOR	OR + NOT	$Y = (A + B)'$	1 only when $A=0, B=0$
XOR	Exclusive OR	$Y = A \oplus B$	1 when inputs differ
XNOR	Exclusive NOR	$Y = (A \oplus B)'$	1 when inputs same

Truth Table for basic 2-input gates:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NAND and NOR are called “Universal Gates” because any Boolean function/any other gate (AND, OR, NOT) can be constructed using only NAND gates or only NOR gates.

1.6 Boolean Algebra

Boolean Algebra is a mathematical system with variables that take only two values (0, 1) and operations AND ($\cdot$), OR ($+$), NOT ($'$).

1.6.1 Basic Postulates/Laws

Law	Expression
Identity	$A+0=A$, $A \cdot 1=A$
Null	$A+1=1$, $A \cdot 0=0$
Idempotent	$A+A=A$, $A \cdot A=A$
Complement	$A+A'=1$, $A \cdot A'=0$
Commutative	$A+B=B+A$, $A \cdot B=B \cdot A$
Associative	$(A+B)+C=A+(B+C)$
Distributive	$A \cdot (B+C)=A \cdot B+A \cdot C$
Absorption	$A+A \cdot B=A$, $A \cdot (A+B)=A$
Involution	$(A')'=A$

1.6.2 DeMorgan’s Theorems

1. $(A+B)' = A' \cdot B'$ (complement of a sum = product of complements)
2. $(A \cdot B)' = A' + B'$ (complement of a product = sum of complements)

These are used to convert AND-OR expressions into NAND/NOR forms.

1.7 K-Map (Karnaugh Map) Simplification

A **K-map** is a graphical method to simplify Boolean expressions/minimize logic circuits, avoiding lengthy algebraic manipulation.

Procedure: 1. Draw a grid with 2^n cells for n variables (2-var $\rightarrow$ 4 cells, 3-var $\rightarrow$ 8 cells, 4-var $\rightarrow$ 16 cells). 2. Fill cells with 1s (for minterms present) as per truth table/SOP expression, using **Gray code** ordering for row/column labels (only 1 bit changes between adjacent cells). 3. Group adjacent 1’s into the **largest possible groups** of size 1, 2, 4, 8, 16 (powers of 2). 4. Groups may wrap around edges (K-map is cyclic). 5. Each group gives one product term; write the term by keeping only variables that don’t change across the group. 6. Sum all product terms for the final simplified **SOP (Sum of Products)** expression. (A similar process using 0’s gives the **POS - Product of Sums** form.)

Example — 3 variable K-map for $F(A,B,C) = \sum m(1,3,5,7)$:

AB\C	0	1
00	0	1
01	0	1
11	0	1
10	0	1

All 1's are in column C=1 $\rightarrow$ simplifies to **F = C**

Rules: Groups must contain only 1's (or only 0's for POS); overlapping groups are allowed; every 1 must be covered by at least one group; prefer larger groups (fewer literals in resulting term).

1.8 Combinational Circuits – Adders & Subtractors

Combinational circuits: output depends only on present inputs (no memory element).

1.8.1 Half Adder

Adds two single bits (A, B); produces Sum and Carry. **No carry-in.**

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = $A \oplus B$, Carry = $A \cdot B$

1.8.2 Full Adder

Adds three bits: A, B, and Carry-in (Cin); produces Sum and Carry-out (Cout). Used to build multi-bit adders.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum = $A \oplus B \oplus \text{Cin}$, Cout = $AB + \text{Cin}(A \oplus B)$

A full adder can be built from **two half adders + one OR gate**.

1.8.3 Subtractor (Half & Full)

Half Subtractor: subtracts B from A (single bit); outputs Difference and Borrow. Difference = $A \oplus B$, Borrow = $A' \cdot B$

Full Subtractor: subtracts with Borrow-in; outputs Difference and Borrow-out. Difference = $A \oplus B \oplus B_{in}$, Borrow-out = $A'B + A'B_{in} + BB_{in}$

(In practice, subtraction in computers is usually implemented via **2's complement addition** rather than dedicated subtractor circuits.)

1.9 Decoders, Encoders, Multiplexers, Demultiplexers

1.9.1 Decoder

Converts n-bit binary input into 2^n unique outputs (only one output line is active/HIGH at a time). Example: 2-to-4 line decoder, 3-to-8 line decoder. Used in memory address decoding, instruction decoding.

1.9.2 Encoder

Performs the reverse of a decoder: converts 2^n input lines (only one active at a time) into an n-bit binary code. Example: 8-to-3 (octal-to-binary) encoder, keyboard encoders. **Priority Encoder:** resolves the case when multiple inputs are active simultaneously by giving priority to the highest-order input.

1.9.3 Multiplexer (MUX)

A “**data selector**”: selects one of many (2^n) input lines and routes it to a single output line, based on select lines. Example: 4-to-1 MUX has 4 data inputs, 2 select lines, 1 output. Used for: data routing, parallel-to-serial conversion, implementing Boolean functions.

1.9.4 Demultiplexer (DEMUX)

Reverse of MUX: takes a single input and routes it to one of 2^n output lines, based on select lines. Used for: serial-to-parallel conversion, data distribution.

Circuit	Inputs	Outputs	Function
Decoder	n	2^n	1-of- 2^n output activation
Encoder	2^n	n	Reverse of decoder
MUX	2^n data + n select	1	Many→One selection
DEMUX	1 data + n select	2^n	One→Many distribution

1.10 Carry Look-Ahead Adder (CLA)

In a simple **Ripple Carry Adder**, each full adder must wait for the carry from the previous stage — this causes propagation delay proportional to the number of bits (slow for large numbers).

The **Carry Look-Ahead Adder** eliminates this delay by calculating all carry bits **in advance**, directly from the input bits, using extra logic gates.

Define for each bit position i : - **Generate**: $G_i = A_i \cdot B_i$ (this stage generates a carry on its own) - **Propagate**: $P_i = A_i \oplus B_i$ (this stage propagates an incoming carry)

Carry equations: - $C_1 = G_0 + P_0 \cdot C_0$ - $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$ - $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

Advantage: Much faster addition (carry doesn't need to ripple through each stage).

Disadvantage: More complex hardware (more gates), cost increases with number of bits.

1.11 Combinational Logic Design

Design procedure for a combinational circuit: 1. Understand the problem statement; determine number of inputs and outputs. 2. Draw the **truth table** relating inputs to outputs. 3. Derive **Boolean (SOP/POS) expressions** for each output from the truth table. 4. **Simplify** the expressions using Boolean algebra or K-map. 5. Draw the **logic diagram** implementing the simplified expression using gates.

This procedure is applied to design adders, subtractors, code converters (e.g., BCD to Excess-3), comparators, etc.

1.12 Flip-Flops (Sequential Building Blocks)

Unlike combinational circuits, **sequential circuits** have memory — output depends on present input **and** past state (stored in flip-flops).

A **Flip-Flop** is a 1-bit memory (bistable multivibrator) element, edge/level-triggered by a clock signal.

Flip-Flop	Description	Characteristic Equation
SR (Set-Reset)	$S=1, R=0 \rightarrow \text{Set}(Q=1);$ $S=0, R=1 \rightarrow \text{Reset}(Q=0);$ $S=R=1 \rightarrow$ invalid/forbidden state	$Q(\text{next}) = S + R'Q$ ($SR=0$)
D (Data/Delay)	Output Q follows input D at clock edge; avoids invalid state of SR	$Q(\text{next}) = D$
JK	Improvement over SR; $J=K=1$ causes toggle (no invalid state)	$Q(\text{next}) = JQ' + K'Q$
T (Toggle)	Single input T ; $T=1$ toggles output, $T=0$ holds state	$Q(\text{next}) = T \oplus Q$

Triggering: Level-triggered (latches) respond as long as clock/enable is active; Edge-triggered flip-flops respond only at the rising/falling edge of the clock (more precise,

used in registers/counters).

1.13 Registers

A **register** is a group of flip-flops used to store multiple bits (a word) of data, with common clock.

Type	Function
Buffer Register	Simply stores data (parallel in-parallel out)
Shift Register	Shifts stored bits left/right on each clock pulse
Serial In Serial Out (SISO)	Data shifted in and out one bit at a time
Serial In Parallel Out (SIPO)	Serial input, all bits available in parallel
Parallel In Serial Out (PISO)	Parallel input, serial output
Parallel In Parallel Out (PIPO)	Parallel input and output
Bidirectional Shift Register	Can shift both left and right

Registers are used to hold data, addresses, or instructions temporarily within the CPU (e.g., Accumulator, PC, MAR, MBR/MDR).

1.14 Counters

A **counter** is a sequential circuit (built from flip-flops) that counts pulses; the count follows a predefined sequence of binary states.

1.14.1 Asynchronous (Ripple) Counter

- Flip-flops are **not** triggered by a common clock; output of one FF acts as clock for the next.
- Simple to design but **slower** (propagation delay accumulates through each stage).
- Example: 4-bit ripple counter counts 0000 to 1111.

1.14.2 Synchronous Counter

- All flip-flops are triggered by the **same clock signal simultaneously**.
- **Faster** than asynchronous (no cumulative delay), but requires more complex combinational logic to decide next state of each FF.
- Example: Synchronous up/down counter, using JK flip-flops with additional gating logic.

Feature	Asynchronous Counter	Synchronous Counter
Clock	Different for each FF (ripple)	Common clock for all FF
Speed	Slower	Faster
Design complexity	Simple	Complex

Feature	Asynchronous Counter	Synchronous Counter
Propagation delay	Cumulative	Minimal (parallel)

Other counter types: Up counter, Down counter, Up/Down counter, Modulo-N counter (counts 0 to N-1 then resets), Ring counter, Johnson (twisted-ring) counter.

1.15 ALU (Arithmetic Logic Unit)

The ALU is the part of the CPU that performs all **arithmetic** (add, subtract, multiply, divide, increment, decrement) and **logical** (AND, OR, NOT, XOR, shift, compare) operations.

- Inputs: two operands (from registers) + control signals (specifying which operation).
- Output: Result + status flags (Zero, Carry, Overflow, Sign).
- Built from combinational circuits: adders, shifters, comparators, and gates, combined with a **multiplexer** to select the required operation output.

1.16 Micro-Operations

A **micro-operation** is an elementary/atomic operation performed on data stored in registers during one clock pulse — the CPU carries out its work as a sequence of micro-operations.

Types: 1. **Register Transfer micro-operations** – move binary data from one register to another (e.g., $R1 \leftarrow R2$). 2. **Arithmetic micro-operations** – perform arithmetic on data in registers (add, subtract, increment, decrement). 3. **Logic micro-operations** – perform bit-wise logical operations (AND, OR, XOR, complement). 4. **Shift micro-operations** – shift data left/right within a register (logical shift, arithmetic shift, circular/rotate shift).

Example notation: $R3 \leftarrow R1 + R2$ means “add contents of R1 and R2, store result in R3.”

1.17 ALU Chip

An **ALU chip** (e.g., the classic 74181 4-bit ALU IC) is a single integrated circuit implementing an arithmetic-logic unit that can perform a set of arithmetic and logic functions selected via function-select input lines (S0-S3), along with mode control (M) to choose arithmetic or logic mode, and carry input/output for cascading multiple chips to build wider ALUs (e.g., cascading four 74181 chips for a 16-bit ALU).

1.18 Faster Algorithms — Multiplication & Division

1.18.1 Booth’s Multiplication Algorithm

An algorithm for multiplying signed binary numbers (2’s complement) efficiently by examining pairs of bits of the multiplier and deciding to **add, subtract, or do nothing**.

ing to a partial product, followed by an arithmetic right shift. Reduces the number of add operations, especially useful for consecutive 1's or 0's runs in the multiplier — this makes multiplication faster than plain shift-and-add.

Rule for each step (examining Q_n and Q_{n-1} , the last bit shifted out):

Q_n	Q_{n-1}	Operation
0	0	No operation, only shift
0	1	Add multiplicand to partial product, then shift
1	0	Subtract multiplicand from partial product, then shift
1	1	No operation, only shift

1.18.2 Array Multiplier

A hardware combinational circuit using an array of AND gates and full adders arranged in rows/columns to compute the product of two binary numbers in parallel (faster than sequential shift-add for hardware implementation).

1.18.3 Division Algorithms

- **Restoring Division:** subtract divisor from partial remainder; if result is negative, restore (add back) the divisor and set quotient bit 0; otherwise set quotient bit 1. Repeat, shifting left each time.
- **Non-Restoring Division:** avoids the “restore” step by conditionally adding or subtracting the divisor depending on the sign of the previous remainder, making it faster than restoring division (no wasted restore cycles).

Unit 2: Basic Organization

2.1 Von Neumann Architecture

Proposed by John von Neumann; the foundation of most modern computers. Key idea: **both program instructions and data are stored in the same memory** and use the same bus for transfer (a “stored program” concept).

Main components: - CPU (Control Unit + ALU + Registers) - Memory Unit (stores both instructions and data together) - Input/Output devices - A **single common bus** connecting these (shared for data and instructions)

Limitation — Von Neumann Bottleneck: since the same bus is used for both instructions and data, the CPU cannot fetch an instruction and read/write data simultaneously, limiting speed.

*(Contrast: **Harvard Architecture** uses separate memories/buses for instructions and data, allowing simultaneous access — used in many microcontrollers/DSPs.)*

2.2 Operational Flow Chart of a Computer

Represents the sequence of steps the CPU follows to execute a program:

Start → Fetch Instruction → Decode Instruction → Fetch Operands
→ Execute Instruction → Store Result → Update PC → (repeat) → Stop

2.3 Instruction Cycle

The **instruction cycle** (fetch-decode-execute cycle) is the sequence of steps a CPU repeats continuously to process instructions.

1. **Fetch Cycle:** CPU fetches the instruction from memory (address in **PC - Program Counter**) into the **IR - Instruction Register**; PC is incremented.
2. **Decode Cycle:** Control Unit decodes the opcode in IR to determine the operation and addressing mode.
3. **Fetch Operand(s):** Operands are fetched from registers/memory as required by the addressing mode.
4. **Execute Cycle:** ALU/CU performs the specified operation.
5. **Store/Write-back:** Result is stored back to register or memory.
6. **Interrupt Check:** CPU checks for pending interrupts before starting the next fetch cycle.

Fetch → Decode → Operand Fetch → Execute → Write-back → Interrupt Check → (loop)

2.4 Organization of the Central Processing Unit (CPU)

The CPU consists of: - **ALU (Arithmetic Logic Unit):** performs computations - **Control Unit (CU):** generates control signals to coordinate all operations - **Registers:** small, fast storage locations, e.g.: - **PC (Program Counter):** holds address of next instruction - **IR (Instruction Register):** holds currently executing instruction - **MAR (Memory Address Register):** holds address to be accessed in memory - **MBR/MDR**

(Memory Buffer/Data Register): holds data being transferred to/from memory - **Accumulator (AC):** holds intermediate arithmetic/logic results - **General Purpose Registers:** used for temporary data storage during execution

2.5 Control Unit Design: Hardwired vs Micro-Programmed

The Control Unit generates the timing and control signals needed to execute instructions.

2.5.1 Hardwired Control Unit

- Control signals generated using **fixed logic circuits** (combinational logic + sequential logic, e.g., using decoders, counters, gates).
- **Fast** (signals produced directly by hardware) but **difficult to modify** — changing the instruction set requires redesigning the circuit.
- Suitable for **RISC** processors (simpler, fixed instruction sets).

2.5.2 Micro-Programmed Control Unit

- Control signals are generated by a **micro-program** stored in a **Control Memory (ROM)**; each instruction corresponds to a sequence of **micro-instructions**.
- **Flexible** — modifying control behavior only needs a change in microcode, not hardware.
- **Slower** than hardwired due to the extra memory-access step for each micro-instruction.
- Suitable for **CISC** processors (large, complex instruction sets).

Feature	Hardwired	Micro-programmed
Speed	Fast	Slower
Flexibility	Low (needs redesign)	High (just change microcode)
Cost	Cheaper for simple ISA	Costlier (needs control memory)
Used in	RISC	CISC

2.6 CPU Organization Types (based on registers)

2.6.1 Single Accumulator Organization

- All computation results pass through a single register — the **Accumulator (AC)**.
- Instruction format: $\text{ADD } X \text{ means } AC \leftarrow AC + M[X]$.
- Simple, but accumulator becomes a bottleneck.

2.6.2 General Register Organization

- Uses **multiple general-purpose registers** instead of a single accumulator, e.g., R1, R2, R3...
- Instructions specify explicit source and destination registers, e.g., $\text{ADD } R1, R2, R3 \text{ means } R1 \leftarrow R2 + R3$.

- Reduces memory references (registers are faster than memory), improves performance; but needs more complex instruction formats to specify multiple register operands.

2.6.3 Stack Organization

- Uses a **LIFO (Last In First Out) stack** instead of named registers; operations implicitly use the top of the stack.
- Instructions are typically **zero-address** — operands are implicitly taken from the stack.
- Example: to compute $(A+B)*C$, values are **pushed** onto the stack, and operators **pop** operands, compute, and **push** the result back (this uses **Reverse Polish/Postfix Notation**).
- Push/Pop operations: **PUSH** places a value on top of stack; **POP** removes the top value.
- Advantage: simpler instruction decoding, no need to specify operand addresses; commonly used in expression evaluation and some CPU designs (e.g., Java Virtual Machine).

2.7 Addressing Modes

Addressing modes specify **how the operand's actual location (effective address) is determined** from the address field of an instruction.

Mode	Description	Example
Immediate	Operand is given directly in the instruction	MOV R1, #5 (R1←5)
Direct (Absolute)	Address field contains the effective address of operand	MOV R1, [1000]
Indirect	Address field contains the address of a memory location that holds the effective address	MOV R1, @[1000]
Register	Operand is in a register itself	MOV R1, R2
Register Indirect	Register holds the address of the operand	MOV R1, (R2)
Displacement (Indexed)	Effective address = Register value + constant offset	MOV R1, 4(R2)
Relative	Effective address = PC + offset (used for branches)	JMP +8
Base Register	Effective address = Base register + offset	Used in segmented memory

Mode	Description	Example
Auto-increment/Auto-decrement	Register is automatically incremented/decremented after/before use (useful for arrays/stacks)	MOV R1, (R2)+

2.8 Instruction Formats

An instruction typically has two parts: **Opcode** (operation code – specifies the operation) and **Operand(s)/Address field(s)**.

Based on number of addresses (operands) specified:

Format	Description	Example
Zero-address	Uses stack; no explicit operand address (implicit)	ADD (pops 2, pushes result)
One-address	Uses accumulator implicitly as second operand & destination	ADD X → AC ← AC + M[X]
Two-address	One operand doubles as source and destination	ADD R1, R2 → R1 ← R1 + R2
Three-address	Separate fields for two source operands and one destination	ADD R1, R2, R3 → R1 ← R2 + R3

Trade-off: fewer addresses per instruction → shorter instructions but more instructions needed (more memory accesses); more addresses → longer instructions but fewer instructions.

2.9 Data Transfer and Manipulation Instructions

- **Data Transfer instructions:** move data between registers, memory, and I/O without changing content, e.g., MOV, LOAD, STORE, PUSH, POP, EXCHANGE.
- **Data Manipulation instructions:** perform operations that change data values, grouped as:
 - **Arithmetic:** ADD, SUB, MUL, DIV, INC, DEC
 - **Logical & Bit manipulation:** AND, OR, NOT, XOR, CLEAR, SET, COMPLEMENT
 - **Shift instructions:** Logical Shift, Arithmetic Shift, Rotate (Circular Shift)

2.10 I/O Organization (Overview)

Refers to how the CPU communicates with peripheral (input/output) devices. (Full detail in Unit 4.) Broadly: - **Programmed I/O** – CPU actively polls/waits for device - **Interrupt-driven I/O** – device interrupts CPU when ready - **DMA (Direct Memory**

Access) – peripheral transfers data directly to/from memory, bypassing CPU for the actual data movement

2.11 Bus Architecture

A **bus** is a set of common wires/lines that connect major components (CPU, memory, I/O) and carry data, addresses, and control signals.

Types of bus lines: - **Data Bus:** carries actual data being transferred (bidirectional)
- **Address Bus:** carries the memory/IO address to be accessed (unidirectional, from CPU) - **Control Bus:** carries control signals (Read/Write, clock, interrupt, bus request/grant)

Bus organization types: - **Single Bus System:** one common bus shared by all units; simple, cheaper, but only one data transfer at a time (bottleneck). - **Multiple Bus System:** separate buses for different transfers (e.g., separate register bus, memory bus); allows simultaneous transfers, faster but more complex/costly.

2.12 Programming (CPU) Registers — Summary

Register	Full Form	Purpose
PC	Program Counter	Holds address of the next instruction to fetch
IR	Instruction Register	Holds the instruction currently being executed
MAR	Memory Address Register	Holds address of memory location to be accessed
MBR / MDR	Memory Buffer/Data Register	Holds data read from/written to memory
AC	Accumulator	Holds intermediate arithmetic/logic operation results
SP	Stack Pointer	Points to the top of the stack
general purpose registers	R0, R1, R2...	Temporary storage during program execution
Status/Flag Register	PSW/Flags	Holds condition codes: Zero, Carry, Sign, Overflow

Unit 3: Memory Organization

3.1 Memory Hierarchy

Since fast memory is expensive and cheap memory is slow, computers use a **hierarchy** of memory types balancing **speed, cost, and capacity**.

Registers (fastest, smallest, costliest)

↓

Cache Memory

↓

Main Memory (RAM)

↓

Auxiliary/Secondary Memory (Hard Disk, SSD)

↓

Magnetic Tape/Backup (slowest, largest, cheapest)

General rule: As we move down the hierarchy → **speed decreases, capacity increases, cost per bit decreases**.

3.2 Main Memory (RAM / ROM)

Main memory is directly accessible by the CPU and holds currently running programs/data.

3.2.1 RAM (Random Access Memory) — Volatile

Type	Description
SRAM (Static RAM)	Stores bits using flip-flops; faster, no need for refreshing, but more expensive and less dense — used for cache memory
DRAM (Dynamic RAM)	Stores bits as charge on a capacitor; needs periodic refreshing ; slower but cheaper & denser — used for main memory

3.2.2 ROM (Read Only Memory) — Non-volatile

Type	Description
ROM	Data written permanently at manufacture time; cannot be modified
PROM (Programmable ROM)	Can be programmed once by the user (using special equipment)
EPROM (Erasable PROM)	Can be erased using UV light and reprogrammed

Type	Description
EEPROM (Electrically Erasable PROM)	Can be erased/reprogrammed electrically, byte by byte
Flash Memory	Similar to EEPROM but faster and erasable in blocks; used in pen drives, SSDs

3.3 Auxiliary (Secondary) Memory

Non-volatile, high-capacity, slower storage used for permanent data storage (outside main memory). Examples: **Magnetic Disks (HDD)**, **Magnetic Tape**, **Optical Disks (CD/DVD)**, **SSD**. Characteristics: large capacity, low cost per bit, slower access than main memory, retains data without power.

3.4 Associative Memory (Content Addressable Memory - CAM)

Unlike normal RAM (accessed by **address**), associative memory is accessed by **content** — you supply a data word (or part of it), and the memory searches **all locations in parallel** and returns the matching location(s)/data.

- Faster searching (parallel comparison) but **more expensive hardware** (needs comparison logic at every memory cell).
- Used in: cache memory (tag matching), database search operations, virtual memory address translation (TLB).

Structure: Argument register (holds search key), Key register (mask - specifies which bits to compare), Match register (indicates matching words).

3.5 Cache Memory

A small, very fast memory placed **between CPU and Main Memory** to reduce the effective memory access time, by holding copies of frequently/recently used data and instructions.

Principle: Locality of Reference - **Temporal locality:** recently accessed data is likely to be accessed again soon. - **Spatial locality:** data near a recently accessed location is likely to be accessed soon.

3.5.1 Hit / Miss Ratio

- **Cache Hit:** requested data found in cache.
- **Cache Miss:** requested data not in cache → must be fetched from main memory.
- **Hit Ratio = Number of Hits / Total Memory Accesses**
- **Average Access Time = Hit ratio × Cache access time + Miss ratio × Main memory access time**

3.5.2 Cache Mapping Techniques

Technique	Description
Direct Mapping	Each main memory block maps to exactly one specific cache line (block address mod number of cache lines). Simple, cheap, but causes more conflicts/misses.
Fully Associative Mapping	A main memory block can go into any cache line. Flexible, fewer misses, but needs complex/costly comparison hardware (associative search).
Set-Associative Mapping	Compromise: cache divided into sets, each set has multiple lines; a block maps to a specific set but can go in any line within that set .

3.5.3 Cache Write Policies

- **Write-through:** every write updates both cache and main memory simultaneously (simple, consistent, but slower).
- **Write-back:** write updates only cache; main memory updated later when the block is replaced (faster, but needs a “dirty bit” to track modified blocks).

3.5.4 Cache Replacement Algorithms (when cache is full)

- **LRU (Least Recently Used):** replace the block not used for the longest time.
- **FIFO (First In First Out):** replace the oldest loaded block.
- **LFU (Least Frequently Used):** replace the block used least often.
- **Random:** replace a randomly chosen block.

3.6 Virtual Memory

A memory management technique that gives programs the illusion of a very large, continuous address space, even if physical (main) memory is smaller — by using secondary storage (disk) as an extension of RAM.

- Programs are divided into fixed-size **pages** (or variable-size **segments**); only needed pages are loaded into main memory (**demand paging**); rest remain on disk.
- **Virtual/Logical Address** (generated by CPU) is translated to a **Physical Address** (actual RAM location) using a **Page Table** (via **MMU - Memory Management Unit**).
- **Page Fault:** occurs when a requested page is not currently in main memory; triggers the OS to load it from disk (may replace another page if memory is full, using a page replacement algorithm like LRU, FIFO, Optimal).

Benefits: allows running programs larger than physical memory, enables multiprogramming, provides memory protection/isolation between processes.

3.7 Memory Management Hardware

- **MMU (Memory Management Unit):** hardware that performs the translation from logical/virtual addresses to physical addresses at run time.
- **Page Table:** data structure (in memory) that stores the mapping of virtual page numbers to physical frame numbers.
- **TLB (Translation Lookaside Buffer):** a small, fast **associative cache** for recently used page table entries — speeds up address translation by avoiding repeated page-table lookups in memory.
- **Segmentation hardware:** uses a **Segment Table** to map logical segments to physical memory (alternative/complement to paging; segments are variable-sized and correspond to logical program units like code, stack, data).

3.8 Magnetic Disk and Its Performance

A magnetic disk (HDD) consists of one or more **platters** coated with magnetic material, divided into **tracks** (concentric circles), and each track divided into **sectors**. Data is read/written by a **read/write head** mounted on an **arm** that moves across the disk surface, while the platter spins on a spindle.

Performance / access time components: - **Seek Time:** time for the read/write head to move to the correct track (largest component of delay). | Component | Meaning | |—|—| | **Seek Time** | Time to position the head over the correct track | | **Rotational Latency (delay)** | Time for the desired sector to rotate under the head | | **Transfer Time** | Time to actually read/write the data once positioned |

Total Access Time = Seek Time + Rotational Latency + Transfer Time

Other terms: **Rotational speed** (measured in RPM, e.g., 7200 RPM), **Data transfer rate**, **Disk capacity** = tracks/platter × sectors/track × bytes/sector × number of platters (surfaces).

3.9 Magnetic Tape

A **sequential-access** storage medium — data is stored on a long magnetic-coated ribbon and must be accessed **in order** (unlike disks, which allow direct/random access).

- Very high storage capacity, low cost per bit — commonly used for **backup and archival** storage.
- Slow access (must wind/rewind to reach the desired data) — not suitable for frequently accessed data.
- Examples: DAT (Digital Audio Tape), LTO (Linear Tape-Open) tape drives.

Feature	Magnetic Disk	Magnetic Tape
Access type	Direct/Random	Sequential
Speed	Faster	Slower
Cost per bit	Higher	Lower
Typical use	Primary/secondary storage	Backup/archival

Unit 4: I/O Organization

4.1 Peripheral Devices

Peripheral devices are hardware connected to the computer to provide input, output, or storage functions, operating at speeds much slower/different than the CPU.

Category	Examples
Input devices	Keyboard, Mouse, Scanner, Joystick
Output devices	Monitor, Printer, Plotter, Speaker
Storage devices	Hard Disk, Pen Drive, CD/DVD
Communication devices	Modem, Network Interface Card

Because peripherals differ greatly in speed, data format, and operating principle compared to the CPU, an **I/O Interface** is required.

4.2 I/O Interface

An **I/O interface** is a hardware circuit that sits between the CPU/system bus and a peripheral device, handling: - **Speed synchronization** between fast CPU and slow devices - **Data format conversion** (e.g., serial $\leftrightarrow$ parallel) - **Error detection** - Providing **status/control/data registers** for CPU to communicate with the device

Each device connected to the CPU needs its own interface (controller) with: - **Data register**: holds data being transferred. - **Status register**: flags indicating device state (ready/busy/error). - **Control register**: holds commands from the CPU to the device.

4.3 Modes of Data Transfer

There are three main methods for transferring data between the CPU/memory and I/O devices:

4.3.1 Programmed I/O

- CPU issues I/O command and then continuously **polls/monitors** the device's status register in a loop, waiting until the device is ready.
- Simple to implement, but **wastes CPU time** (busy-waiting), since CPU can do nothing else during the transfer.

4.3.2 Interrupt-Driven I/O

- CPU issues the I/O command and continues executing other tasks; the device sends an **interrupt signal** to the CPU when it is ready/transfer is complete.
- CPU then temporarily suspends its current task, executes an **Interrupt Service Routine (ISR)** to handle the transfer, and resumes its earlier task.
- More efficient than programmed I/O (no busy-waiting), but has overhead of context switching for every data item (not ideal for large/bulk transfers).

4.3.3 DMA (Direct Memory Access)

- For bulk data transfer, a special hardware controller (**DMA controller**) directly transfers data **between I/O device and memory**, without CPU intervention for each byte/word.
- CPU only initiates the transfer (gives starting address, word count, direction) and is interrupted only **once**, at the end of the entire block transfer.
- Much faster for large transfers (disk, network) since CPU is freed for other tasks during the transfer.

Mode	CPU Involvement	Speed	Best for
Programmed I/O	CPU actively polls	Slow	Small, simple devices
Interrupt-driven I/O	CPU interrupted per transfer/data unit	Moderate	Devices with occasional, unpredictable data
DMA	CPU involved only at start/end	Fast	Bulk transfers (disk, memory-to-memory)

4.4 Priority Interrupt

When **multiple devices** can raise interrupts simultaneously, the CPU needs a way to decide which interrupt to service first — this is handled by an **interrupt priority scheme**.

Methods for establishing priority: 1. **Software Polling:** CPU polls each device (in priority order) via a program to check which one requested the interrupt — simple but slow (software overhead). 2. **Daisy Chaining (Hardware, serial priority):** Devices are connected in a series (chain); the interrupt acknowledge signal passes through each device in order; the first device (highest priority, closest to CPU) that is requesting service intercepts the signal and blocks it from passing further. Simple hardware, but priority is fixed by physical position. 3. **Parallel Priority Interrupt (using priority encoder):** each device has its own interrupt request/acknowledge line; a priority encoder circuit determines the highest-priority active request and generates the corresponding vector/address — faster than daisy chaining, needs more hardware.

Vectored vs Non-vectored interrupts: - **Vectored interrupt:** device itself supplies the address of its ISR (interrupt service routine) directly to CPU. - **Non-vectored interrupt:** CPU has to poll/determine the ISR address itself (fixed location or by software polling).

4.5 Direct Memory Access (DMA) — Detailed

DMA Controller (DMAC) components/registers: - **Address Register:** holds the memory address for the transfer (auto-incremented after each word). - **Word Count Register:** holds the number of words remaining to transfer (decremented after

each word; transfer ends when it reaches 0). - **Control Register:** specifies mode (read/write), and transfer type.

DMA transfer steps: 1. CPU initializes DMA controller with starting address, word count, and direction (read/write). 2. DMA controller requests **bus control** from CPU (Bus Request - BR). 3. CPU grants bus control (Bus Grant - BG) and pauses its own bus usage (**cycle stealing**, or CPU may halt completely for **burst mode** transfer). 4. DMA controller directly transfers data between device and memory. 5. On completion, DMA controller sends an **interrupt** to the CPU, and control of the bus returns to CPU.

DMA transfer modes: - **Burst Mode:** entire block transferred in one go; CPU is halted for the whole duration (fast, but CPU idle for longer). - **Cycle Stealing Mode:** DMA controller transfers one word at a time, "stealing" individual bus cycles between CPU operations (CPU only slightly delayed, better CPU utilization).

4.6 Input Output Processor (IOP)

An **IOP** (also called a **Channel** in mainframe systems) is a specialized processor dedicated to managing I/O operations independently of the CPU.

- Executes its own **I/O programs** (stored in main memory) to control data transfer between memory and peripherals, communicating with the CPU only to start operations and report completion (via interrupts).
- Frees the main CPU almost entirely from I/O handling — a further evolution beyond DMA (DMA handles one transfer; IOP can manage and sequence multiple/complex I/O operations autonomously).
- Common in large mainframe/high-performance systems.

4.7 Serial Communication

Data is transmitted **one bit at a time**, sequentially, over a single communication line (as opposed to parallel communication which sends multiple bits simultaneously over multiple lines).

- **Advantages:** fewer wires needed, less signal interference/crosstalk, suitable for **long-distance** transmission.
- **Disadvantage:** slower than parallel transmission per unit time (though modern high-speed serial links now outperform parallel for many applications due to reduced timing-skew issues).

Types: - **Synchronous serial transmission:** data sent in blocks/frames with a shared clock signal (or embedded clock/timing) to keep sender and receiver synchronized; more efficient for large data blocks. - **Asynchronous serial transmission:** no shared clock; each data unit (character) is framed with **Start bit** and **Stop bit(s)** (and optional parity bit) to allow the receiver to synchronize independently for each character. Simpler, but has more overhead per character.

4.8 I/O Controllers

An **I/O controller** is the hardware unit (chip/circuit) that manages a specific device or a class of devices, handling low-level details (electrical signaling, buffering, error checking) and providing a standard interface to the CPU/system bus. Examples: Disk Controller, Keyboard Controller, USB Controller, Display Controller.

4.9 Asynchronous Data Transfer

Refers to data transfer between two units (e.g., CPU and I/O device) that **do not share a common clock** — timing is instead coordinated using **handshaking signals** or **strobe pulses**, since the two units may operate at different, unpredictable speeds.

4.9.1 Strobe Control

A single control signal (**strobe pulse**) is used by the source (or destination) to indicate when data is valid/should be read — a simple one-way timing signal.

- **Source-initiated strobe:** source unit places data on the bus and then activates the strobe signal to inform the destination that data is ready to be read.
- **Destination-initiated strobe:** destination activates the strobe to request data from the source, which then puts the valid data on the bus.

Limitation: no confirmation/acknowledgement is sent back — the source doesn't know if the destination actually received the data.

4.9.2 Handshaking

Improves on strobe control by using **two control signals** — one from each unit — so both units confirm actions to each other before proceeding, ensuring reliable transfer.

Example (Source-initiated handshaking): 1. Source places data on bus, then sets “**Data Valid**” signal. 2. Destination, after accepting data, sets “**Data Accepted**” signal. 3. Source, sensing “Data Accepted,” removes data and disables “Data Valid.” 4. Destination disables “Data Accepted,” ready for next transfer.

This two-way exchange of control signals ensures that transfer proceeds only when **both** sides are ready — making handshaking more reliable than a simple strobe, though it requires more signal lines and time.

Feature	Strobe Control	Handshaking
Control signals	1	2 (Valid + Accepted/Response)
Acknowledgement	No	Yes
Reliability	Lower	Higher
Complexity	Simple	Slightly more complex

Unit 5: Process Organization

5.1 Basic Concept of Microprocessors

5.1.1 8085 Microprocessor (8-bit)

Intel 8085 is an **8-bit** microprocessor with a **16-bit address bus** (can address 64 KB of memory: $2^{16} = 65536$ locations) and **8-bit data bus**.

Registers of 8085: | Register | Description | |—|—| | **A (Accumulator)** | 8-bit; holds one operand and stores the result of ALU operations | | **B, C, D, E, H, L** | General purpose 8-bit registers; can be paired as **BC, DE, HL** for 16-bit operations | | **HL pair** | Often used as a memory pointer (indirect addressing) | | **PC (Program Counter)** | 16-bit; holds address of next instruction | | **SP (Stack Pointer)** | 16-bit; points to top of stack | | **Flag Register** | 8-bit; holds status flags: **S**(Sign), **Z**(Zero), **AC**(Auxiliary Carry), **P**(Parity), **CY**(Carry) |

Basic architecture blocks: ALU, Register array, Instruction decoder & control unit, Timing and control circuitry, Interrupt control, Serial I/O control.

5.1.2 8086 Microprocessor (16-bit)

Intel 8086 is a **16-bit** microprocessor with a **20-bit address bus** (can address 1 MB of memory) and **16-bit data bus**.

Key features: - Internally divided into two functional units working in parallel (a simple pipelining concept): - **BIU (Bus Interface Unit):** fetches instructions, handles address calculations and bus/memory access. - **EU (Execution Unit):** decodes and executes instructions, contains ALU and general registers. - Uses **instruction queue (pipelining):** BIU pre-fetches instructions into a queue while EU executes previous ones, improving throughput. - **Segmented memory model:** memory divided into four segments — **Code Segment (CS), Data Segment (DS), Stack Segment (SS), Extra Segment (ES)** — Physical Address = Segment $\times$ 16 + Offset. - Registers: **AX, BX, CX, DX** (general purpose, each divisible into 8-bit halves AH/AL etc.), **SI, DI** (index registers), **BP, SP** (pointer registers), **CS, DS, SS, ES** (segment registers), **IP** (Instruction Pointer), **FLAGS**.

Feature	8085	8086
Data bus	8-bit	16-bit
Address bus	16-bit	20-bit
Memory addressable	64 KB	1 MB
Pipelining	No	Yes (BIU + EU, instruction queue)
Registers	8-bit only	16-bit, segment registers

5.2 Assembly Instruction Set (8085) — Common Categories

Category	Example Instructions	Function
Data Transfer	MOV, MVI, LDA, STA, LXI, LDAX, STAX	Move/load/store data between registers/memory
Arithmetic	ADD, ADI, SUB, SUI, INR, DCR, INX, DCX	Arithmetic operations
Logical	ANA, ORA, XRA, CMP, CMA, RAL, RAR	Logical/compare/rotate operations
Branching	JMP, JZ, JNZ, JC, JNC, CALL, RET	Change program flow, conditional/unconditional
Stack, I/O & Machine control	PUSH, POP, IN, OUT, HLT, NOP, EI, DI	Stack operations, I/O, machine control

Common addressing modes used in 8085 instructions: Immediate (MVI A, 05H), Direct (LDA 2000H), Register (MOV A,B), Register Indirect (MOV A,M — uses HL as pointer), Implied (CMA).

5.3 Assembly Language Programs (8085)

5.3.1 Addition of Two 8-bit Numbers

```
LDA 2000H    ; Load first number into A from memory
MOV B, A     ; Move it to register B
LDA 2001H    ; Load second number into A
ADD B        ; A = A + B
STA 2002H    ; Store result in memory
HLT          ; Stop
```

5.3.2 Subtraction of Two 8-bit Numbers

```
LDA 2000H    ; Load minuend into A
MOV B, A     ; Move to B
LDA 2001H    ; Load subtrahend into A
SUB B        ; A = A - B
STA 2002H    ; Store result (difference)
HLT
```

5.3.3 Block Transfer (Copying a Block of Data)

Transfers a block of N bytes from a source memory location to a destination.

```
LXI H, 2000H ; HL = source address
LXI D, 3000H ; DE = destination address
MVI C, 05H   ; C = counter (number of bytes to transfer)
BACK: MOV A, M ; Get byte from source (pointed by HL)
      STAX D   ; Store byte to destination (pointed by DE)
      INX H    ; Increment source pointer
```

```

    INX D    ; Increment destination pointer
    DCR C    ; Decrement counter
    JNZ BACK ; Repeat if counter  $\neq$  0
    HLT

```

5.3.4 Finding the Greatest Number in a Block

```

LXI H, 2000H    ; HL = pointer to array
MVI C, 04H      ; C = (count of numbers - 1) comparisons
MOV A, M        ; A = first number (assume largest so far)
BACK: INX H     ; Point to next number
      CMP M     ; Compare A with next number
      JNC SKIP  ; If A  $\geq$  M, no change (carry not set)
      MOV A, M  ; Else, update A with the larger number
SKIP: DCR C
      JNZ BACK
      STA 3000H ; Store the greatest number
      HLT

```

5.3.5 Table Search (Linear Search for a Key)

```

LXI H, 2000H    ; HL = pointer to array
MVI C, 05H      ; C = number of elements
MVI A, 07H      ; A = key to search
BACK: CMP M     ; Compare key with current element
      JZ FOUND
      INX H
      DCR C
      JNZ BACK
      MVI A, 00H ; Not found flag
      STA 3000H
      JMP DONE
FOUND: MVI A, 01H ; Found flag
      STA 3000H
DONE: HLT

```

5.3.6 Numeric Manipulation Example (BCD to Binary / 16-bit addition, etc.)

Example — **16-bit addition** (numeric manipulation on register pairs):

```

LHLD 2000H      ; Load first 16-bit number into HL
XCHG            ; Move it to DE
LHLD 2002H      ; Load second 16-bit number into HL
DAD D           ; HL = HL + DE (16-bit add)
SHLD 2004H      ; Store 16-bit result
HLT

```

(General exam tip: In every program — initialize pointers/registers first, use a loop with a counter and conditional jump for repetitive tasks, and always end with HLT.)

5.4 Introductory Concept of Pipelining

Pipelining is a technique that overlaps the execution of multiple instructions, similar to an assembly line — different stages of several instructions are processed **simultaneously** in different pipeline stages, improving overall instruction throughput (though not the latency of a single instruction).

Typical 5-stage instruction pipeline: 1. **IF** - Instruction Fetch 2. **ID** - Instruction Decode 3. **OF/EX** - Operand Fetch / Execute 4. **MEM** - Memory Access 5. **WB** - Write Back

Clock:	1	2	3	4	5	6
Instr1:	IF	ID	EX	MEM	WB	
Instr2:		IF	ID	EX	MEM	WB
Instr3:			IF	ID	EX	MEM WB

Without pipelining, each instruction would need 5 clock cycles sequentially; with pipelining, throughput approaches **1 instruction completed per clock cycle** (after the pipeline is filled).

Pipeline Hazards (problems that reduce efficiency): | Hazard | Cause | |—|—| |
Structural Hazard | Hardware resource conflict (two stages need the same resource simultaneously) | | **Data Hazard** | An instruction depends on the result of a previous instruction still in the pipeline | | **Control Hazard** | Caused by branch instructions — pipeline doesn't know which instruction to fetch next until branch is resolved |

5.5 Flynn's Classification

Proposed by Michael Flynn (1966); classifies computer architectures based on the number of **instruction streams** and **data streams** processed simultaneously.

Class	Full Form	Description	Example
SISD	Single Instruction, Single Data	Traditional sequential (von Neumann) computer — one instruction operates on one data item at a time	Classic uniprocessor systems
SIMD	Single Instruction, Multiple Data	One instruction operates simultaneously on multiple data elements	Vector processors, GPUs, array processors
MISD	Multiple Instruction, Single Data	Multiple instructions operate on the same data stream (rare in practice)	Fault-tolerant systems (redundant computation for safety)

Class	Full Form	Description	Example
MIMD	Multiple Instruction, Multiple Data	Multiple processors execute different instructions on different data independently	Multiprocessor/multicore systems, distributed systems

5.6 Feng's Classification

Proposed by Tse-yun Feng; classifies computer architectures based on the **degree of parallelism**, measured by counting the number of bits processed per unit time, using two parameters: - **Word length (n)**: number of bits processed in parallel within a word (word-level parallelism). - **Bit length (m)**: number of words processed simultaneously (bit-level parallelism, i.e., number of parallel processing elements).

Feng represents this as a plot/pair (n, m) and defines four categories:

Category	Word (Serial/Parallel)	Bit (Serial/Parallel)	Description
WSBS (Word Serial, Bit Serial)	Serial	Serial	Processes 1 bit of 1 word at a time (slowest)
WSBP (Word Serial, Bit Parallel)	Serial	Parallel	Processes all bits of 1 word at a time — most conventional computers
WPBS (Word Parallel, Bit Serial)	Parallel	Serial	Processes 1 bit each of multiple words simultaneously
WPBP (Word Parallel, Bit Parallel)	Parallel	Parallel	Processes multiple full words simultaneously (fully parallel, fastest) — array/vector processors

5.7 Parallel Architecture Classification (Based on Memory Access)

Type	Description
UMA (Uniform Memory Access)	All processors share a common physical memory with equal access time for all — typical symmetric multiprocessor (SMP) system
NUMA (Non-Uniform Memory Access)	Each processor has local memory (fast access) plus access to other processors' memory (slower); access time varies by location
COMA (Cache Only Memory Architecture)	Distributed memory is used entirely as large caches; no fixed home location for data
Shared Memory Multiprocessor	All processors access a single shared address space
Distributed Memory Multiprocessor	Each processor has its own private memory; processors communicate via message passing (interconnection network)

5.8 Concept of Multi-Core Architecture

A **multi-core processor** contains **two or more independent processing units (cores)** on a single chip, each capable of executing instructions independently, enabling true parallel execution of multiple threads/processes.

Key features: - Each core typically has its own **L1 cache**; cores may share **L2/L3 cache**. - Cores communicate via an on-chip **interconnect/bus**. - Improves overall system throughput and multitasking performance without needing to increase clock speed (which is limited by power/heat constraints).

Benefits: better performance per watt, improved multitasking, exploits **thread-level parallelism (TLP)**. **Challenges:** software must be written to exploit parallelism (multi-threading); cache coherence must be maintained across cores; increased design complexity.

Difference from pipelining: Pipelining overlaps stages of **sequential instruction execution within a single core**, whereas multi-core architecture achieves parallelism by having **multiple independent cores** execute separate instruction streams **simultaneously** — the two techniques are complementary and are typically used together in modern processors.